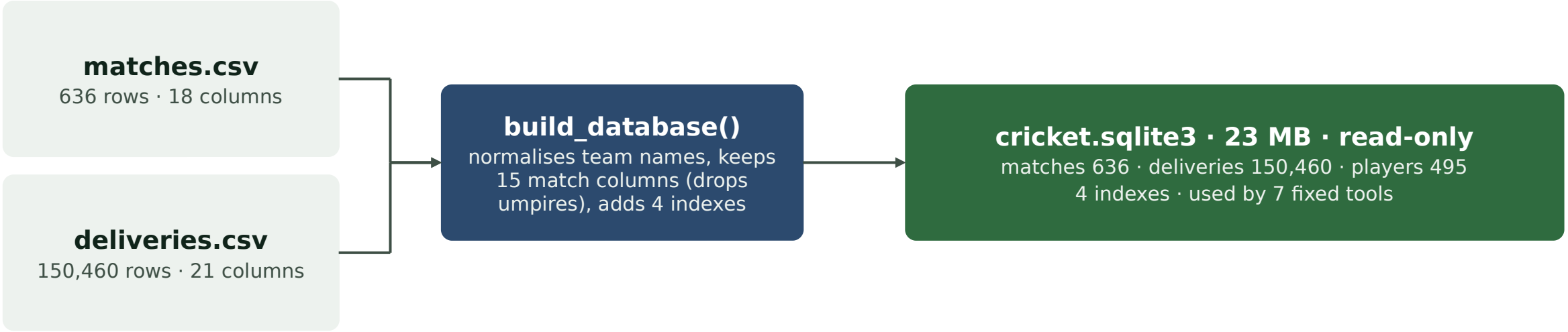
Inside Crickbot's data

Tables, chunks and vectors: real queries run on the real databases

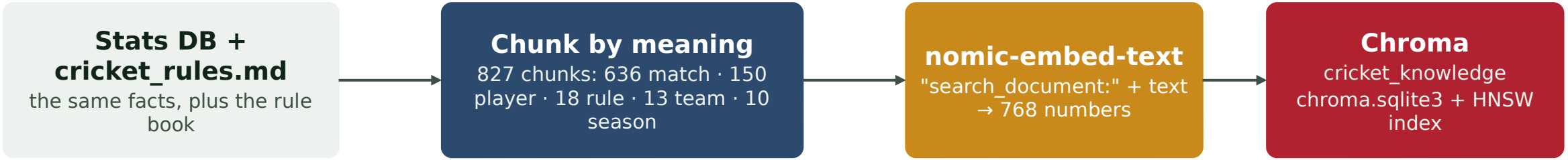
10 IPL seasons (2008–2017) · cricket.sqlite3 (stats) · chroma.sqlite3 (vectors). Every screenshot is real output, captured on this laptop.

From two CSV files to two databases

Path 1 · facts (exact numbers)

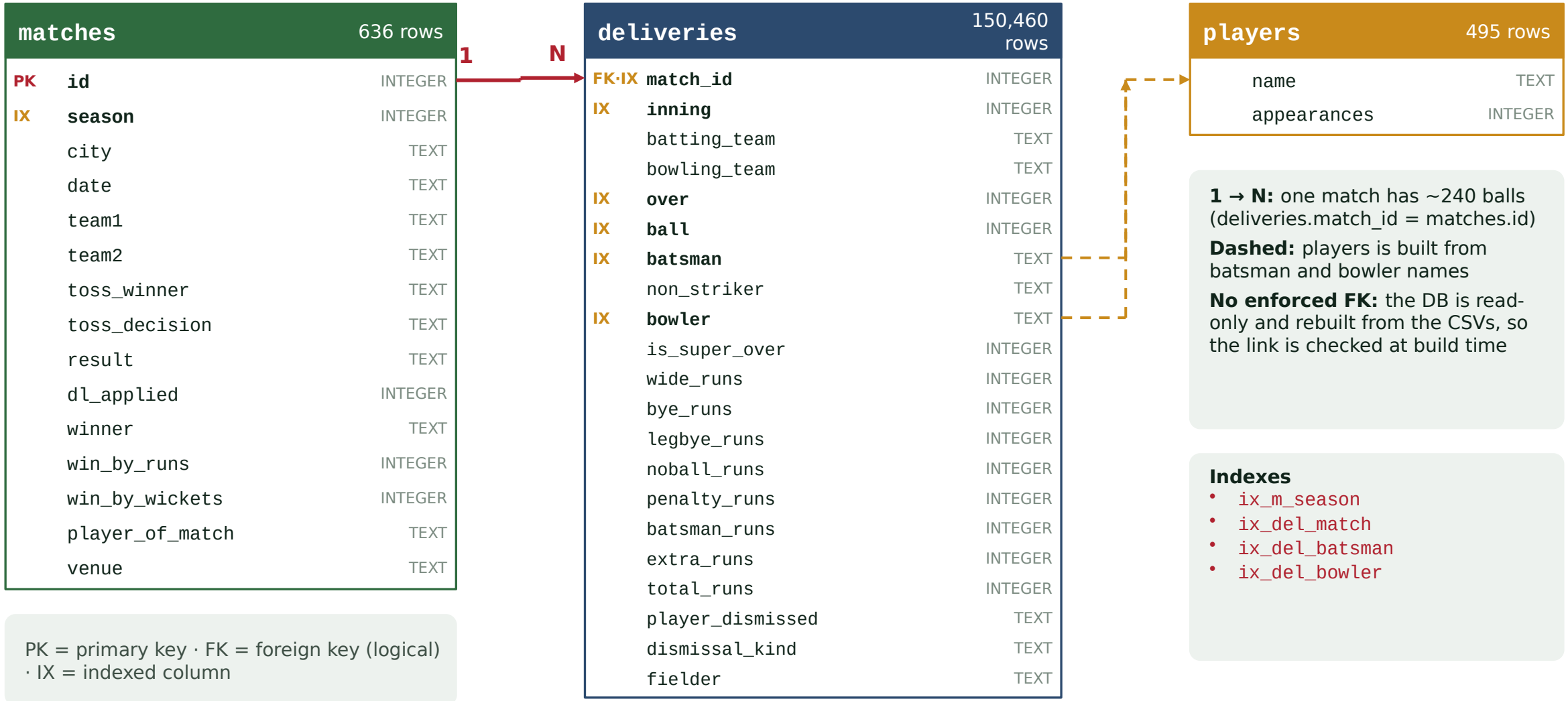


Path 2 · meaning (search by similarity)



Built by `scripts/setup.sh`. Re-run `python -m cricket_chatbot.ingest` and only chunks whose hash changed are embedded again.

Entity relationship diagram: cricket.sqlite3



PK = primary key · FK = foreign key (logical) · IX = indexed column

1 → N: one match has ~240 balls (deliveries.match_id = matches.id)

Dashed: players is built from batsman and bowler names

No enforced FK: the DB is read-only and rebuilt from the CSVs, so the link is checked at build time

Indexes

- `ix_m_season`
- `ix_del_match`
- `ix_del_batsman`
- `ix_del_bowler`

Tables in the stats database, with row counts

cricket.sqlite3 · Tables in the stats database, with row counts

```
sqlite> SELECT 'matches' AS table_name, COUNT(*) AS rows FROM matches
   ...> UNION ALL SELECT 'deliveries', COUNT(*) FROM deliveries
   ...> UNION ALL SELECT 'players', COUNT(*) FROM players;
```

table_name	rows
matches	636
deliveries	150460
players	495

(3 rows)

636 matches and 150,460 balls from 10 IPL seasons, built from the two CSV files.

One row in matches: the 2017 IPL final

cricket.sqlite3 · One row in matches: the 2017 IPL final

```
sqlite> SELECT id, season, date, team1, team2, winner, win_by_runs,
   ...>        player_of_match
   ...> FROM matches WHERE id = 59;
```

id	season	date	team1	team2	winner	win_by_runs	player_of_match
59	2017	2017-05-21	Mumbai Indians	Rising Pune Supergiant	Mumbai Indians	1	KH Pandya

(1 row)

One row per match. Mumbai Indians won the final by 1 run.

Ball-by-ball rows in deliveries for that match

cricket.sqlite3 · Ball-by-ball rows in deliveries for that match

```
sqlite> SELECT inning, over, ball, batsman, bowler, batsman_runs AS runs,
   ...>        extra_runs AS extras, player_dismissed AS out
   ...> FROM deliveries WHERE match_id = 59
   ...> ORDER BY inning, over, ball LIMIT 8;
```

inning	over	ball	batsman	bowler	runs	extras	out
1	1	1	LMP Simmons	JD Unadkat	0	0	
1	1	2	LMP Simmons	JD Unadkat	0	0	
1	1	3	LMP Simmons	JD Unadkat	0	0	
1	1	4	LMP Simmons	JD Unadkat	1	0	
1	1	5	PA Patel	JD Unadkat	1	0	
1	1	6	LMP Simmons	JD Unadkat	1	0	
1	2	1	LMP Simmons	Washington Sundar	0	0	
1	2	2	LMP Simmons	Washington Sundar	1	0	

(8 rows)

One row per ball. This is the data the live match replays.

A join: the final's innings, rebuilt from its balls

cricket.sqlite3 · A join: each innings of the final, rebuilt from its balls

```
sqlite> SELECT d.inning, d.batting_team,
   ...>        SUM(d.total_runs) AS runs,
   ...>        SUM(d.player_dismissed <> '') AS wickets
   ...> FROM deliveries d JOIN matches m ON m.id = d.match_id
   ...> WHERE m.id = 59 GROUP BY d.inning;
```

inning	batting_team	runs	wickets
1	Mumbai Indians	129	8
2	Rising Pune Supergiant	128	6

(2 rows)

129/8 and 128/6: the real scorecard, computed from 248 ball rows.

Top 5 run scorers across all 10 seasons

cricket.sqlite3 · Top 5 run scorers across all 10 seasons

```
sqlite> SELECT batsman, SUM(batsman_runs) AS runs,
   ...>        COUNT(DISTINCT match_id) AS matches
   ...> FROM deliveries GROUP BY batsman
   ...> ORDER BY runs DESC LIMIT 5;
```

batsman	runs	matches
SK Raina	4548	157
V Kohli	4423	141
RG Sharma	4207	153
G Gambhir	4132	146
DA Warner	4014	114

(5 rows)

Aggregated over 150,460 rows in milliseconds, thanks to the index on batsman.

Champion of every season

cricket.sqlite3 · Champion of every season

```
sqlite> SELECT season, winner AS champion, team1 || ' vs ' || team2 AS final
   ...> FROM matches m
   ...> WHERE id = (SELECT id FROM matches WHERE season = m.season
   ...>             ORDER BY date DESC, id DESC LIMIT 1)
   ...> ORDER BY season;
```

season	champion	final
2008	Rajasthan Royals	Chennai Super Kings vs Rajasthan Royals
2009	Deccan Chargers	Deccan Chargers vs Royal Challengers Bangalore
2010	Chennai Super Kings	Chennai Super Kings vs Mumbai Indians
2011	Chennai Super Kings	Chennai Super Kings vs Royal Challengers Bangalore
2012	Kolkata Knight Riders	Chennai Super Kings vs Kolkata Knight Riders
2013	Mumbai Indians	Mumbai Indians vs Chennai Super Kings
2014	Kolkata Knight Riders	Kings XI Punjab vs Kolkata Knight Riders
2015	Mumbai Indians	Mumbai Indians vs Chennai Super Kings
2016	Sunrisers Hyderabad	Sunrisers Hyderabad vs Royal Challengers Bangalore
2017	Mumbai Indians	Mumbai Indians vs Rising Pune Supergiant

(10 rows)

The last match of each season is its final: what the season_summary tool uses.

The players table (built from deliveries)

cricket.sqlite3 · The players table (built from deliveries)

```
sqlite> SELECT name, appearances FROM players
   ...> ORDER BY appearances DESC LIMIT 5;
```

name	appearances
SK Raina	4293
JH Kallis	4090
SR Watson	3910
V Kohli	3758
RG Sharma	3616

(5 rows)

495 players. Used to turn 'Kohli' into the scorecard name 'V Kohli'.

How the data was cut into chunks

match	636	One per match: teams, venue, toss, scores, result, top batters and bowlers. Finals are labelled "final".
player	150	The 150 most active players: teams, batting and bowling stats.
rule	18	One per section of cricket_rules.md (no ball, LBW, DRS…).
team	13	Each franchise: matches played, matches won, titles.
season	10	Each year: matches, champion, final, top 5 run scorers and wicket takers.

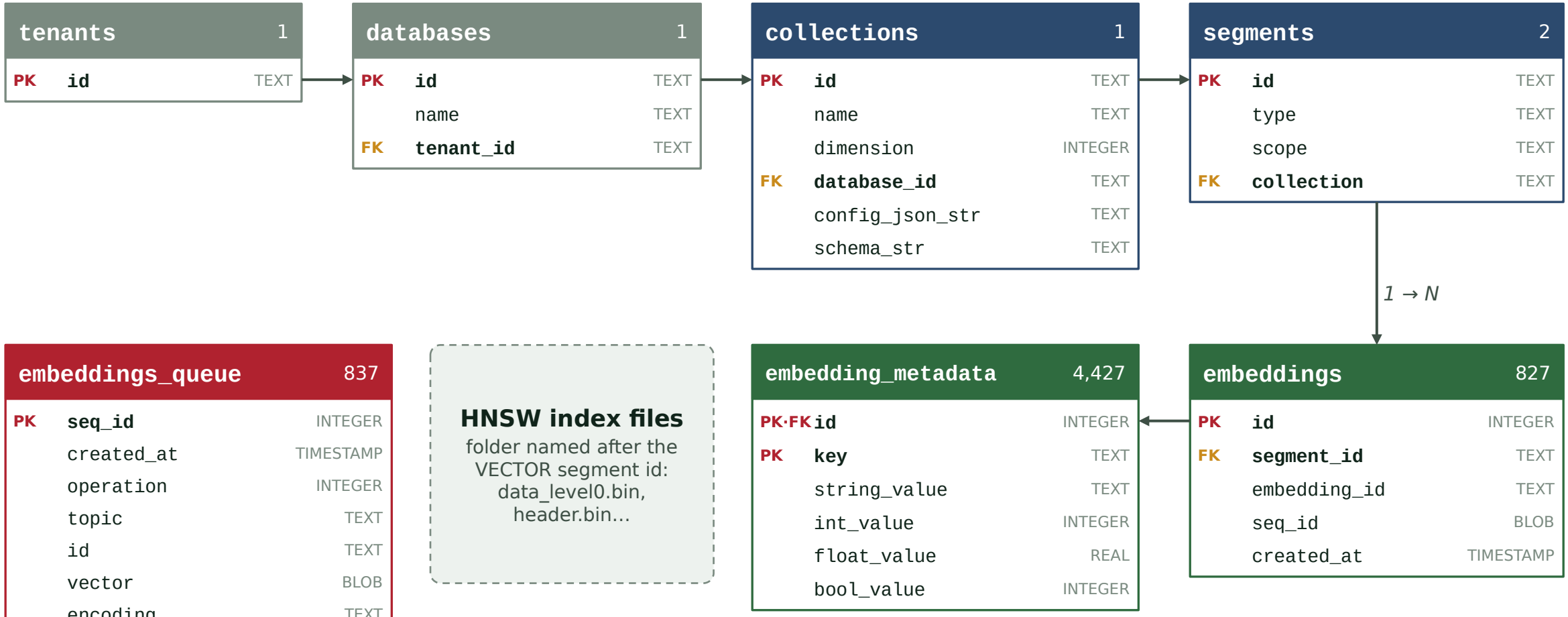
= 827 chunks · one chunk = one complete fact

Why by meaning, not by size?

- Cutting every 500 characters splits a scorecard in half; the half that is found can't answer the question.
- Each chunk is short (well under the model's limit), so it is embedded in one piece.
- Metadata (type, season, match_id, teams, stage) lets search filter before ranking: "2016 final" only looks at 2016.
- A hash per chunk: re-ingest skips chunks that didn't change.

`Code: backend/src/cricket_chatbot/rag/documents.py`

Entity relationship diagram: inside Chroma



Read it as: one collection (cricket_knowledge, 768 dims) → 2 segments (METADATA in SQLite, VECTOR in HNSW files) → 827 chunks → ~5 metadata keys each (document text, type, season, hash…). The queue logs every write, including the vector BLOB.

Chroma's own tables, with row counts

chroma.sqlite3 · Chroma's own tables, with row counts

```
sqlite> SELECT 'collections' AS table_name, COUNT(*) AS rows FROM collections
   ...> UNION ALL SELECT 'segments', COUNT(*) FROM segments
   ...> UNION ALL SELECT 'embeddings', COUNT(*) FROM embeddings
   ...> UNION ALL SELECT 'embedding_metadata', COUNT(*) FROM embedding_metadata
   ...> UNION ALL SELECT 'embeddings_queue', COUNT(*) FROM embeddings_queue;
```

table_name	rows
collections	1
segments	2
embeddings	827
embedding_metadata	4427
embeddings_queue	837

(5 rows)

827 chunks. The write queue has 837 entries: the 10 season finals were re-embedded after they were labelled.

The collection and its vector size

chroma.sqlite3 · The collection and its vector size

```
sqlite> SELECT name, dimension FROM collections;
```

name	dimension
cricket_knowledge	768

(1 row)

One collection, cricket_knowledge. Every vector has 768 numbers.

Chunks per document type

chroma.sqlite3 · Chunks per document type

```
sqlite> SELECT string_value AS type, COUNT(*) AS chunks
   ...> FROM embedding_metadata WHERE key = 'type'
   ...> GROUP BY string_value ORDER BY chunks DESC;
```

type	chunks
match	636
player	150
rule	18
team	13
season	10

(5 rows)

Chunked by meaning: one chunk per match, player, rule, team and season.

What a chunk looks like (stored text)

chroma.sqlite3 · What a chunk looks like (stored text)

```
sqlite> SELECT e.embedding_id AS chunk_id, m.string_value AS text
   ...> FROM embeddings e
   ...> JOIN embedding_metadata m ON m.id = e.id AND m.key = 'chroma:document'
   ...> WHERE e.embedding_id IN ('player-v-kohli', 'season-2016',
   ...>                          'rule-no-ball-and-free-hit');
```

chunk_id	text
rule-no-ball-and-free-hit	No ball and free hit. A no ball is called most often when the bowler's front foot lands fully b…
season-2016	IPL 2016 season: 60 matches. Champion: Sunrisers Hyderabad, beating Royal Challengers Bangalore…
player-v-kohli	V Kohli played IPL cricket for Royal Challengers Bangalore. Batting 2008-2017: 4423 runs from 3…

(3 rows)

Each chunk is one complete fact, short enough to embed in one piece.

Where the vector is stored

chroma.sqlite3 · Where the vector is stored

```
sqlite> SELECT id AS chunk_id, length(vector) AS vector_bytes, encoding
   ...> FROM embeddings_queue
   ...> WHERE id IN ('player-v-kohli', 'season-2016', 'rule-no-ball-and-free-hit');
```

chunk_id	vector_bytes	encoding
rule-no-ball-and-free-hit	3072	FLOAT32
season-2016	3072	FLOAT32
player-v-kohli	3072	FLOAT32

(3 rows)

3,072 bytes = 768 numbers × 4 bytes (float32) per chunk.

The 768 numbers stored for one chunk

chroma (Python API) · The 768 numbers for one chunk

```
>>> col = chromadb.PersistentClient('var/chroma').get_collection('cricket_knowledge')
... v = col.get(ids=['player-v-kohli'], include=['embeddings'])['embeddings'][0]
... print(len(v), v[:8])
```

chunk_id	dimensions	first 8 numbers
player-v-kohli	768	[-0.0366, 0.0366, -0.1858, 0.0302, 0.0346, -0.0117, 0.0194, -0.0085, …]

(1 row)

Made by nomic-embed-text. Similar meaning → similar numbers.

A similarity search on the vectors

chroma (Python API) · A similarity search: the question becomes 768 numbers too

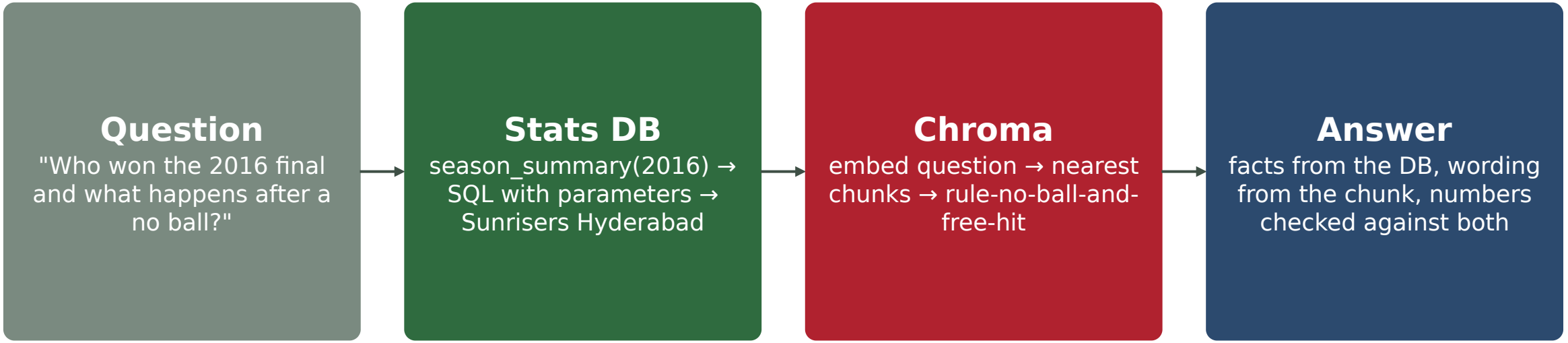
```
>>> qv = embed('search_query: What happens after a no ball?')        # 768 numbers
... col.query(query_embeddings=[qv], n_results=4)
```

rank	chunk_id	distance	text
1	rule-no-ball-and-free-hit	0.264	No ball and free hit. A no ball is called most often when th…
2	rule-byes-and-leg-byes	0.335	Byes and leg byes. Byes are runs taken when the ball passes …
3	rule-ways-of-getting-out	0.36	Ways of getting out. A batter can be out bowled, caught, leg…
4	rule-leg-before-wicket-lbw	0.375	Leg before wicket (LBW). A batter is out lbw when the ball w…

(4 rows)

Smaller distance = closer meaning. The free-hit rule comes first.

How one question uses both databases



Exact numbers come from SQL tables. **Fuzzy meaning** comes from vectors. The model never writes SQL; any number the data can't back is removed.